RotaExpress

Kubernetes na prática

Publicar, expor, testar carga e escalar
uma aplicação com 1, 2 e 3 Pods

Uma transportadora calcula a ordem de pequenas entregas com um serviço chamado RotaExpress. O programa funciona quando um atendente faz um pedido por vez. A dúvida aparece quando vários atendentes usam o serviço ao mesmo tempo: a capacidade atual é suficiente? Quantas cópias da aplicação são necessárias?

A aplicação já está pronta. A infraestrutura não. Neste roteiro você vai fazer o trabalho de quem opera o serviço: preparar o cluster, empacotar a aplicação, escrever os manifestos, publicar, verificar, provocar uma falha de implantação e voltar atrás, rodar o teste de carga dentro do cluster e comparar configurações. Todos os comandos são digitados por você, na ordem em que um profissional os executaria.

A pergunta da investigação

Mantendo a mesma demanda, como a quantidade de Pods afeta o número de rotas concluídas, o tempo de resposta e as falhas? Qual é o custo em recursos de cada configuração?

O que será produzido

- Uma pasta `k8s/` com os manifestos escritos por você: Namespace, Deployment, Service e Job.
- Os registros do k6 de cada execução, salvos em `resultados/`.
- Uma tabela comparativa de 1, 2 e 3 Pods e uma análise curta que justifique uma decisão operacional.

Sistemas suportados. Windows 10/11 com PowerShell e Ubuntu 22.04 ou mais recente. Os comandos de `docker`, `minikube` e `kubectl` são idênticos nos dois. Nos poucos pontos em que o terminal muda, o roteiro mostra as duas versões lado a lado.

Lucca Abbado Neres

Centro Universitário Fundação Assis Gurgacz – FAG

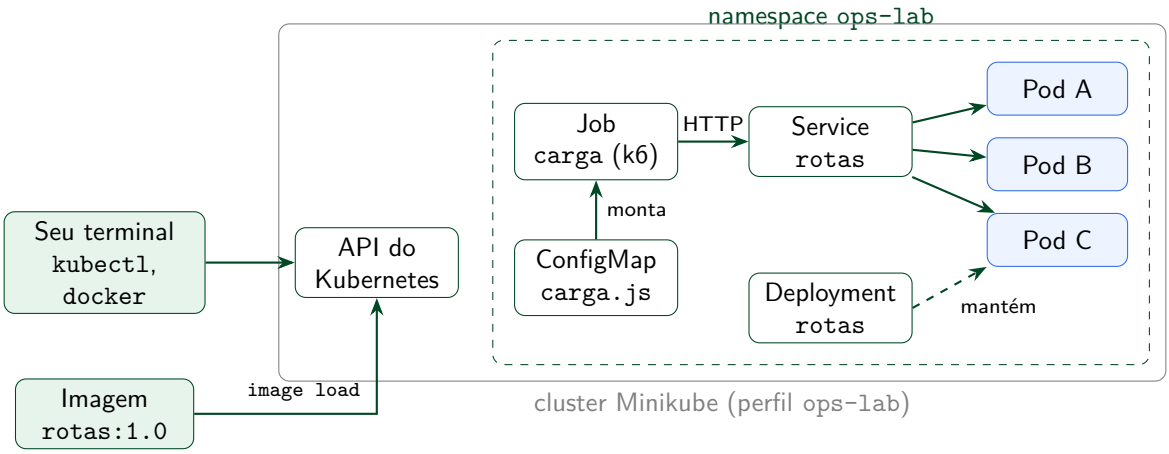
2026

PARTE A

Visão geral antes de digitar qualquer coisa

A.1 O que vai existir quando terminarmos

O desenho abaixo é o estado final do laboratório. Cada caixa é algo que você vai criar com um comando ou com um arquivo YAML. Vale a pena voltar a ele sempre que um passo parecer solto.



Em palavras: o `kubectl` conversa com a API do cluster. Você entrega à API descrições do que quer (manifestos). O Deployment mantém os Pods com a aplicação. O Service dá um nome estável, `rotas`, e distribui conexões entre os Pods prontos. O teste de carga roda como um Job dentro do mesmo cluster, lendo o script k6 de um ConfigMap e acessando `http://rotas:8000`.

A.2 Por que os passos estão nesta ordem

Cada etapa depende de algo que a anterior produziu. Essa cadeia de dependências é a mesma de uma empresa; o que muda lá é a escala e quem executa cada parte (às vezes uma pipeline, às vezes uma pessoa).

Grupo	Tema	Produz	Depende de
0	Ferramentas	`docker`, `kubectl` e `minikube` funcionando	–
1	Cluster e contexto	Cluster ligado, namespace criado, `kubectl` apontando para ele	Ferramentas
2	Imagem	Imagem `rotas:1.0` testada e disponível no nó	Docker
3	Deployment	Pods da aplicação mantidos pelo cluster	Cluster + imagem
4	Service	Nome estável e balanceamento entre Pods	Pods com labels
5	Falha e rollback	Prática de reverter uma implantação ruim	Deployment + Service
6	Teste de carga	ConfigMap, Job do k6 e uma taxa calibrada	Service funcionando
7	Escala e medição	Série de medições com 1, 2 e 3 Pods	Teste calibrado
8	Diagnóstico e decisão	Leitura de falhas, análise e encerramento	Medições

Duas ideias organizam tudo:

1. **Separar domínios de falha.** Testamos a imagem com Docker antes de levá-la ao cluster. Testamos o Service por dentro antes de colocar carga nele. Quando algo quebra, você sabe em qual camada procurar.
2. **Mudar uma variável por vez.** Na medição, só a quantidade de Pods muda. Imagem, taxa, duração e recursos por Pod ficam iguais. Sem isso, a comparação não diz nada.

A.3 Como ler este roteiro

Cada grupo começa com o contexto geral e depois desce ao detalhe de cada comando. Para cada comando há três explicações: **o que faz**, **por que agora** (o motivo de ele vir naquele ponto) e, quando útil, **o que observar** na saída.

- Blocos com barra verde são comandos. Cada comando é uma única linha; quando o papel não comporta, a continuação aparece com o símbolo ↪, mas você digita tudo na mesma linha.
- Blocos com barra azul e linhas numeradas são arquivos YAML que você vai digitar no editor.
- Blocos com moldura fina mostram saídas de exemplo. Nomes de Pods, IPs e tempos serão diferentes na sua máquina.
- Os quadros **Na entrevista** trazem perguntas comuns em processos seletivos de DevOps/SRE ligadas ao passo que você acabou de executar.

Digite os comandos e os manifestos. Copiar e colar funciona, mas a atividade foi pensada para você ler cada campo enquanto digita. Erros de indentação no YAML e nomes trocados são exatamente o tipo de problema que aparece no trabalho, e aprender a reconhecê-los faz parte do exercício.

A.4 Arquivos fornecidos e arquivos que você vai escrever

Arquivo	Situação
`app.py`	Fornecido. A aplicação em Python. Não precisa ser lida em detalhe.
`Dockerfile`, `.dockerignore`	Fornecidos. Descrevem a imagem; você vai ler o Dockerfile no Grupo 2.
`carga.js`	Fornecido. O script do k6; você vai ler só os parâmetros no Grupo 6.
`resultados_modelo.csv`	Fornecido. Tabela vazia para registrar as medições.
`k8s/namespace.yaml`	Você escreve no Grupo 1.
`k8s/deployment.yaml`	Você escreve no Grupo 3, em duas versões.
`k8s/service.yaml`	Você escreve no Grupo 4.
`k8s/job-carga.yaml`	Você escreve no Grupo 6.

O que a aplicação faz, em uma frase. Recebe um pedido em `/api/rota?paradas=9`, calcula por força bruta o menor circuito entre um depósito e nove pontos ($9! = 362.880$ ordens possíveis, trabalho real de CPU) e responde com a rota e o nome do Pod que atendeu. Cada Pod faz um cálculo por vez; se chegar outro pedido enquanto calcula, responde HTTP 503 na hora. Não há fila nem cache. Os endereços `/healthz` e `/readyz` respondem rápido e servem para as verificações de saúde.

GRUPO 0

Ferramentas

Faça antes da aula. Sem isso, nada do resto funciona.

Contexto. Você vai usar três programas. O **Docker** constrói imagens e também serve de base para o cluster local. O **Minikube** cria um cluster Kubernetes de um nó dentro de um contêiner Docker. O **kubectl** é o cliente oficial da API do Kubernetes; é a ferramenta que você usaria do mesmo jeito em um cluster da AWS, do Google ou da empresa. O k6, que gera a carga, não precisa ser instalado: ele vai rodar como contêiner dentro do cluster.

O cluster vai reservar 3 CPUs e 3 GiB de memória; 8 GB de RAM no computador é o mínimo razoável.

0.1 Instalar

Windows (PowerShell)

1. Instale o Docker Desktop com o backend WSL 2 e abra o aplicativo.
2. Em um PowerShell comum (não precisa ser administrador):

```
winget install -e --id Kubernetes.kubectl
winget install -e --id Kubernetes.minikube
```

3. Feche e abra o PowerShell para o PATH ser atualizado.

Ubuntu (terminal)

1. Instale o Docker Engine pelo repositório oficial. Depois, permita usar o Docker sem `sudo` (saia e entre na sessão em seguida):

```
sudo usermod -aG docker $USER
```

2. kubectl e Minikube:

```
sudo snap install kubectl --classic
curl -LO https://storage.googleapis.com/
    ↪ minikube/releases/latest/
    ↪ minikube_latest_amd64.deb
sudo dpkg -i minikube_latest_amd64.deb
```

0.2 Verificar

```
docker version
kubectl version --client
minikube version
```

O que faz. Cada comando imprime a versão do programa. O primeiro também consulta o serviço do Docker: se aparecer a seção `Server` sem erro, o Docker está ativo.

Por que agora. Confirmar as ferramentas antes de criar o cluster separa problema de instalação de problema de Kubernetes. É a mesma lógica de checar a rede antes de culpar a aplicação.

O que observar. No `docker version`, um erro como `cannot connect to the Docker daemon` significa que o Docker Desktop está fechado (Windows) ou que o serviço está parado ou seu usuário não está no grupo `docker` (Ubuntu). Não contorne isso com `sudo`: o Minikube com driver Docker se recusa a rodar como root.

0.3 Abrir a pasta do projeto

Extraia o ZIP e abra um terminal dentro da pasta `projeto`. No Windows, clique com o botão direito na pasta e escolha *Abrir no Terminal*. No Ubuntu, *Abrir no Terminal* pelo gerenciador de arquivos ou `cd` até ela. Todos os comandos do roteiro supõem que o terminal está nessa pasta. Para editar os YAML, use um editor com destaque de sintaxe, como o VS Code.

Critério para avançar. as três versões aparecem sem erro e o terminal está na pasta `projeto`.

GRUPO 1

Cluster e contexto

Criar o lugar onde a aplicação vai rodar e garantir que cada comando chegue nele.

Contexto geral. Um cluster Kubernetes é um conjunto de máquinas (nós) coordenado por um *control plane*, que expõe uma API. Tudo o que você faz no Kubernetes é uma chamada a essa API: criar, ler, alterar ou apagar objetos. O `kubectl` é só um cliente dessa API.

Para saber *qual* API chamar, o `kubectl` lê um arquivo chamado **kubeconfig** (`~/.kube/config` no Ubuntu, `%USERPROFILE%\.kube\config` no Windows). Esse arquivo guarda **contextos**: cada contexto junta um cluster, um usuário e, opcionalmente, um namespace padrão. Um profissional costuma ter vários contextos (desenvolvimento, homologação, produção) no mesmo arquivo. Rodar um comando no contexto errado é um dos incidentes mais comuns e mais evitáveis da área.

Neste grupo, a ordem é: criar o cluster, confirmar para onde o `kubectl` aponta, olhar o que já existe lá dentro, criar o espaço da atividade (namespace) e só então ligar o componente de métricas.

1.1 Criar o cluster

```
minikube start -p ops-lab --driver=docker --cpus=3 --memory=3072
```

O que faz. Cria (ou liga, se já existir) um cluster de um nó chamado `ops-lab`. `-p ops-lab` define o *perfil*, que é o nome do cluster local. `-driver=docker` faz o nó ser um contêiner Docker. `-cpus` e `-memory` (em MiB) limitam o que o nó pode usar do seu computador.

Por que agora. É a infraestrutura. Nada pode ser publicado antes de existir uma API para receber os manifestos.

O que observar. A primeira execução baixa a imagem do nó e os componentes do Kubernetes; pode levar alguns minutos. No final, o Minikube informa que configurou o `kubectl` para usar o contexto `ops-lab`.

1.2 Fixar o perfil para os próximos comandos do Minikube

```
minikube profile ops-lab
```

O que faz. Torna `ops-lab` o perfil padrão do Minikube. A partir daqui, `minikube image load` e `minikube addons` agem nesse cluster sem precisar repetir `-p ops-lab`.

Por que agora. Se você já tem outro cluster Minikube (de outra disciplina, por exemplo), isso evita carregar a imagem no cluster errado.

1.3 Confirmar para onde o kubectl aponta

```
kubectl config get-contexts
kubectl config current-context
```

O que faz. O primeiro lista todos os contextos do kubeconfig e marca com `*` o atual. O segundo imprime só o atual.

Por que agora. Antes de criar ou apagar qualquer coisa, você confirma o alvo. Em empresa, esse é um hábito diário.

```
CURRENT   NAME      CLUSTER   AUTHINFO   NAMESPACE
*         ops-lab   ops-lab   ops-lab    default
```

Na entrevista 1

Pergunta: *Como você evita executar um comando no cluster de produção por engano?*

Uma boa resposta: Primeiro, conferir o contexto com `kubectl config current-context` antes de comandos que alteram estado. Na prática, as equipes usam várias camadas: kubeconfigs separados por ambiente, prompt do terminal que mostra o contexto atual, ferramentas como `kubectx`, permissões RBAC restritas em produção (a maioria das pessoas só lê) e, principalmente, mudanças em produção feitas por pipeline ou GitOps, e não à mão.

1.4 Verificar se a API e o nó respondem

```
kubectl cluster-info
kubectl get nodes -o wide
```

O que faz. `cluster-info` mostra o endereço da API. `get nodes` lista os nós; `-o wide` acrescenta colunas como IP interno, sistema operacional e runtime de contêineres.

Por que agora. Um nó `NotReady` faria os Pods ficarem pendentes mais adiante, e você acharia que o problema é o manifesto.

```
NAME      STATUS   ROLES           AGE   VERSION   INTERNAL-IP    OS-IMAGE             CONTAINER-RUNTIME
ops-lab   Ready    control-plane   90s   v1.3x.x   192.168.49.2   Ubuntu 22.04.5 LTS   docker://2x.x.x
```

1.5 Olhar o que já roda no cluster

```
kubectl get pods -A
```

O que faz. Lista Pods de todos os namespaces (`-A` é abreviação de `-all-namespaces`).

Por que agora. O cluster recém-criado não está vazio. No namespace `kube-system` estão os componentes do próprio Kubernetes: `kube-apiserver` (a API), `etcd` (o banco onde o estado desejado fica guardado), `kube-scheduler` (escolhe em qual nó cada Pod roda), `kube-controller-manager` (os controladores que reconciliam estado), `coredns` (resolve nomes como `rotas`) e `kube-proxy` (implementa os Services no nó). Ver essas peças concretiza o diagrama de arquitetura da aula.

1.6 Criar o namespace da atividade

Crie a pasta dos manifestos. O comando é o mesmo nos dois sistemas:

```
mkdir k8s
```

No editor, crie o arquivo `k8s/namespace.yaml`:

```
1 apiVersion: v1
2 kind: Namespace
3 metadata:
4   name: ops-lab
```

Todo manifesto tem pelo menos estes campos. `apiVersion` diz qual versão da API trata o objeto. `kind` diz o tipo. `metadata.name` é o nome, único dentro do tipo (e do namespace, para objetos que vivem em um). A maioria dos objetos tem ainda um `spec`, o estado desejado; o Namespace não precisa.

```
kubectl apply -f k8s/namespace.yaml
kubectl get namespaces
```

O que faz. `apply -f` envia o arquivo à API. Se o objeto não existe, é criado; se existe, é atualizado para ficar igual ao arquivo. `get namespaces` confirma o resultado.

Por que agora. Os objetos da atividade vão morar nesse namespace. Ele precisa existir antes de qualquer um deles.

Imperativo ou declarativo?

Daria para fazer o mesmo com `kubectl create namespace ops-lab`. Esse é o estilo *imperativo*: você manda executar uma ação. Com `apply -f arquivo.yaml`, o estilo é *declarativo*: você descreve o resultado e a API decide o que fazer. Rodar o mesmo `apply` duas vezes não dá erro, porque o estado já é o descrito. Em empresa, o arquivo vai para o Git, passa por revisão e é aplicado por uma pipeline. É por isso que vamos escrever arquivos.

1.7 Tornar ops-lab o namespace padrão do contexto

```
kubectl config set-context --current --namespace=ops-lab
kubectl config get-contexts
```

O que faz. Grava no contexto atual que o namespace padrão é `ops-lab`. Depois disso, `kubectl get pods` equivale a `kubectl get pods -n ops-lab`.

Por que agora. Sem isso, você teria de escrever `-n ops-lab` em cada comando, e esquecer uma vez faria o objeto ir parar no namespace `default`. Na saída do segundo comando, a coluna `NAMESPACE` agora mostra `ops-lab`.

Na entrevista 2

Pergunta: *Namespaces isolam aplicações? Posso usar um namespace por cliente e considerar que estão seguros?*

Uma boa resposta: Namespace é uma divisão organizacional e de nomes, não uma fronteira de segurança por si só. Por padrão, um Pod de um namespace consegue falar com Services de outro. O isolamento vem de outros mecanismos aplicados por namespace: RBAC (quem pode fazer o quê), NetworkPolicy (quem fala com quem na rede), ResourceQuota e LimitRange (quanto cada um pode consumir) e Pod Security Admission. Para isolamento forte entre clientes, muitas empresas usam clusters separados.

1.8 Habilitar as métricas de uso

```
minikube addons enable metrics-server
```

O que faz. Instala o `metrics-server`, componente que coleta o uso de CPU e memória de nós e Pods. É ele que alimenta o `kubectl top`.

Por que agora. Ele leva um ou dois minutos para produzir as primeiras leituras. Ligando agora, as métricas estarão prontas quando forem necessárias no Grupo 7. Em clusters de empresa, ele (ou algo equivalente, como Prometheus) normalmente já está instalado; o autoscaling horizontal (HPA) depende dele.

Critério para avançar. `kubectl get nodes` mostra o nó `Ready`, `kubectl config get-contexts` mostra `ops-lab` com namespace `ops-lab`, e o namespace aparece em `kubectl get namespaces`.

GRUPO 2

Imagem

O Kubernetes não executa código-fonte. Executa imagens.

Contexto geral. O cluster só sabe iniciar contêineres a partir de imagens. Em uma empresa, o caminho costuma ser: a pipeline de CI constrói a imagem, envia (*push*) para um registry (Docker Hub, GitHub Container Registry, ECR, Harbor) e o nó baixa (*pull*) quando um Pod precisa dela.

No laboratório não há registry. O nó do Minikube tem seu próprio armazenamento de imagens, separado do Docker do seu computador. Por isso a ordem será: construir com Docker, testar com Docker, e só então carregar a imagem dentro do nó com `minikube image load`. Esse último passo faz o papel do push/pull.

2.1 Ler o Dockerfile

```
1 FROM python:3.12-slim
2 WORKDIR /app
3 COPY app.py .
4 ENV PYTHONUNBUFFERED=1 PYTHONDONTWRITEBYTECODE=1
5 USER 10001:10001
6 EXPOSE 8000
7 CMD ["python", "app.py"]
```

Linha 1: parte de uma imagem oficial com Python, versão enxuta. Linha 2: diretório de trabalho dentro da imagem. Linha 3: copia a aplicação. Linha 4: `PYTHONUNBUFFERED` faz o `print` sair imediatamente nos logs, o que importa para o `kubectl logs`; a outra variável evita gravar arquivos `.pyc`. Linha 5: o processo roda com um usuário sem privilégios, não como root. Linha 6: documenta a porta. Linha 7: o processo principal do contêiner.

A aplicação não tem dependências externas, por isso não há `pip install`. O `.dockerignore` garante que só `app.py` e o `Dockerfile` entrem no contexto de build.

2.2 Construir a imagem

```
docker build -t rotas:1.0 .
docker image ls rotas
```

O que faz. `docker build` executa o Dockerfile e produz uma imagem. `-t rotas:1.0` dá nome (`rotas`) e tag (`1.0`). O ponto final é o *contexto de build*: a pasta cujos arquivos o Docker pode copiar. `docker image ls rotas` confirma que a imagem existe e mostra seu tamanho.

Por que agora. A imagem é o artefato que vai para o cluster. Todo o resto do roteiro depende dela.

Na entrevista 3

Pergunta: *Por que não usar a tag `latest` em produção?*

Uma boa resposta: Porque `latest` não diz qual versão está rodando. Duas máquinas podem ter imagens diferentes com o mesmo nome, e o rollback fica impossível de descrever ("voltar para qual `latest`?"). No Kubernetes, imagens com `latest` recebem por padrão `imagePullPolicy: Always`, o que faz cada Pod novo baixar de novo e possivelmente pegar outra versão. O recomendado é uma tag por versão (`1.0`, `1.1` ou o hash do commit) e, quando é preciso garantia total, referenciar pelo digest (`rotas@sha256:...`), que é imutável.

2.3 Testar a imagem fora do Kubernetes

```
docker run -d --rm -p 8000:8000 --name rotas-teste rotas:1.0
```

O que faz. Inicia um contêiner a partir da imagem. `-d` libera o terminal (roda em segundo plano), `-rm` apaga o contêiner quando ele parar, `-p 8000:8000` liga a porta 8000 do seu computador à porta 8000 do contêiner e `-name` dá um nome para referenciá-lo depois.

Agora teste o endereço de saúde e um cálculo:

Windows (PowerShell)

```
curl.exe http://localhost:8000/healthz
curl.exe "http://localhost:8000/api/rota?
    ↪ paradas=9"
```

Ubuntu (terminal)

```
curl http://localhost:8000/healthz
curl "http://localhost:8000/api/rota?
    ↪ paradas=9"
```

No PowerShell use `curl.exe`, com a extensão: em algumas versões, `curl` sozinho é um apelido para outro comando, com saída diferente. Você também pode abrir http://localhost:8000 no navegador.

```
docker logs rotas-teste
docker stop rotas-teste
```

O que faz. `docker logs` mostra o que o processo escreveu na saída padrão. Cada cálculo gera uma linha como `aceito calculo_ms=231.4 total_aceitos=1 total_recusados=0`. `docker stop` encerra o contêiner, e o `-rm` o remove.

Por que agora. Se a imagem não funciona aqui, não vai funcionar no cluster. Testar neste ponto separa "a aplicação está quebrada" de "configurei o Kubernetes errado". Repare também no valor de `calculo_ms`: é o tempo de um cálculo na sua máquina sem limite de CPU. Ele vai ajudar a entender a capacidade de um Pod.

2.4 Carregar a imagem dentro do nó

```
minikube image load rotas:1.0
minikube image ls
```

O que faz. Copia a imagem do Docker do seu computador para o armazenamento de imagens do nó `ops-lab`. O segundo comando lista as imagens que o nó enxerga; procure `docker.io/library/rotas:1.0` na lista.

Por que agora. O Pod vai ser iniciado *pelo nó*, não pelo seu Docker. Sem este passo, o Pod tentaria baixar `rotas:1.0` do Docker Hub e falharia.

Na entrevista 4

Pergunta: *Em um cluster de verdade, como a imagem chega até o nó? E se o registry for privado?*

Uma boa resposta: A pipeline publica a imagem em um registry e o kubelet de cada nó faz o pull quando um Pod precisa dela, conforme a `imagePullPolicy`. Se o registry exige autenticação, cria-se um Secret do tipo `docker-registry` e ele é referenciado em `imagePullSecrets` no Pod (ou na ServiceAccount). Em nuvem, é comum a própria identidade do nó ter permissão de leitura no registry da conta.

Critério para avançar. o `curl` devolveu uma rota, o contêiner de teste foi parado e `rotas:1.0` aparece em `minikube image ls`.

GRUPO 3
Deployment
Declarar quantas cópias devem existir e deixar o cluster cuidar delas.

Contexto geral. Você não cria Pods diretamente. Você cria um **Deployment**, que descreve um modelo de Pod (*template*) e uma quantidade (`replicas`). O Deployment cria um **ReplicaSet**, e o ReplicaSet cria e repõe os Pods. Controladores dentro do control plane comparam o tempo todo o estado desejado com o observado e agem para aproximá-los. É isso que se chama *reconciliação*.

Vamos escrever o Deployment em duas passadas. A primeira tem só o mínimo para a aplicação subir; ela deixa clara a estrutura do manifesto e a hierarquia Deployment, ReplicaSet e Pod. A segunda acrescenta o que uma empresa exigiria antes de aceitar o manifesto: recursos, verificações de saúde e restrições de segurança. Aplicar a segunda versão por cima da primeira também mostra, de graça, como funciona uma atualização gradual (*rolling update*).

3.1 Escrever a primeira versão

Crie `k8s/deployment.yaml`:

```
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: rotas
5    namespace: ops-lab
6    labels:
7      app: rotas
8  spec:
9    replicas: 1
10   selector:
11     matchLabels:
12       app: rotas
13   template:
14     metadata:
15       labels:
16         app: rotas
17     spec:
18       containers:
19         - name: rotas
20           image: rotas:1.0
21           imagePullPolicy: IfNotPresent
22           ports:
23             - containerPort: 8000
```

Lendo o arquivo por blocos.

- Linhas 1–7: identidade. `apps/v1` é o grupo de API dos Deployments. O `namespace` explícito no arquivo evita depender do contexto de quem aplica.
- Linha 9: quantas cópias manter.
- Linhas 10–12: o `selector` diz quais Pods pertencem a este Deployment: os que têm a label `app: rotas`.
- Linhas 13–24: o `template`, isto é, o molde de cada Pod. As labels do template (linhas 15–16) *precisam* satisfazer o selector; se não satisfizerem, a API recusa o manifesto.
- Linhas 18–24: o contêiner. `imagePullPolicy: IfNotPresent` diz ao nó para usar a imagem local se ela já estiver lá, que é o nosso caso. `containerPort` é informativo; documenta a porta em que o

processo escuta.

> YAML usa **espaços**, nunca tabulação. A indentação define a hierarquia: `containers` precisa estar dentro do `spec` do template, e não do `spec` do Deployment. Um erro como `found character that cannot start any token` quase sempre é uma tabulação; `unknown field` quase sempre é um campo no nível errado.

3.2 Aplicar e acompanhar a implantação

```
kubectl apply -f k8s/deployment.yaml
kubectl rollout status deployment/rotas
```

O que faz. O `apply` registra o Deployment na API. O `rollout status` fica esperando até a implantação terminar (todas as réplicas atualizadas e prontas) e então devolve o terminal.

Por que agora. O `apply` só diz que a API *aceitou* o pedido, não que a aplicação subiu. Em uma pipeline, o `rollout status` é o passo que faz o deploy falhar se os Pods não ficarem prontos. Sem ele, um deploy quebrado "passaria".

3.3 Ver a hierarquia que foi criada

```
kubectl get deployment,replicaset,pods
kubectl get pods -o wide --show-labels
```

O que faz. O primeiro lista os três tipos de objeto de uma vez. O segundo mostra os Pods com IP, nó e labels.

Por que agora. Você pediu um objeto e apareceram três. Ler os nomes mostra quem criou quem.

```
NAME                    READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/rotas   1/1     1            1           40s

NAME                               DESIRED   CURRENT   READY   AGE
replicaset.apps/rotas-6d4b8c9f7d   1         1         1       40s

NAME                          READY   STATUS    RESTARTS   AGE
pod/rotas-6d4b8c9f7d-x2k9p    1/1     Running   0          40s
```

O que observar. O ReplicaSet tem o nome do Deployment mais um hash do template (`6d4b8c9f7d`). O Pod tem o nome do ReplicaSet mais um sufixo aleatório. Quando o template mudar, o hash muda e nasce um ReplicaSet novo. Anote o IP do Pod mostrado pelo `-o wide`; ele vai ser útil no próximo passo e no Grupo 4.

3.4 Inspecionar um Pod

Copie o nome do seu Pod da saída anterior.

```
kubectl describe pod rotas-6d4b8c9f7d-x2k9p
kubectl logs deployment/rotas
```

O que faz. `describe` mostra tudo o que o cluster sabe sobre o objeto, em texto legível: imagem, portas, condições, e no final a seção **Events**, com o histórico do que aconteceu (agendado no nó, imagem presente, contêiner criado, iniciado). `logs deployment/rotas` mostra a saída de um Pod desse Deployment; aqui, a linha `RotaExpress em http://localhost:8000`.

Por que agora. São as duas primeiras ferramentas de diagnóstico de qualquer profissional. Vale conhecê-las com tudo funcionando, para reconhecer depois o que é diferente quando algo quebrar. Lembre: `describe` e Events mostram o que o *Kubernetes* viu; `logs` mostram o que a *aplicação* escreveu.

3.5 Apagar um Pod de propósito

```
kubectl delete pod -l app=rotas
kubectl get pods -o wide
```

O que faz. `-l app=rotas` seleciona por label, então você não precisa do nome exato. O Pod é encerrado. O segundo comando mostra que já existe outro, com outro nome e, muito provavelmente, outro IP.

Por que agora. Este é o comportamento central do Deployment: o desejado é 1, o observado ficou 0, o ReplicaSet criou um novo. Também mostra um problema que o Grupo 4 resolve: o IP do Pod mudou, então ninguém deveria depender dele.

Se quiser ver a troca acontecendo, rode `kubectl get pods -w` em outro terminal antes de apagar; `-w` (*watch*) fica imprimindo as mudanças até você apertar `Ctrl+C`.

Na entrevista 5

Pergunta: *Por que criar um Deployment em vez de criar o Pod diretamente?*

Uma boa resposta: Um Pod criado sozinho não é recriado se for apagado, se o processo for removido do nó ou se o nó cair. O Deployment, por meio do ReplicaSet, mantém a quantidade desejada, permite escalar com um campo e faz atualizações graduais com histórico de revisões e rollback. Pods avulsos ficam para casos pontuais, como um contêiner temporário de depuração.

3.6 Segunda versão: recursos, saúde e segurança

Edite `k8s/deployment.yaml`. Acrescente, dentro do contêiner, logo depois de `ports`, os blocos abaixo. A indentação de `resources`, `readinessProbe`, `livenessProbe` e `securityContext` é a mesma de `ports`: dez espaços. O arquivo completo está no Apêndice B.

```
25          resources:
26            requests:
27              cpu: 500m
28              memory: 64Mi
29            limits:
30              cpu: 500m
31              memory: 128Mi
32          readinessProbe:
33            httpGet:
34              path: /readyz
35              port: 8000
36            periodSeconds: 5
37          livenessProbe:
38            httpGet:
39              path: /healthz
40              port: 8000
41            initialDelaySeconds: 10
42            periodSeconds: 10
43          securityContext:
44            runAsNonRoot: true
45            allowPrivilegeEscalation: false
46            readOnlyRootFilesystem: true
```

Recursos (linhas 25–31). `requests` é o que o Pod declara precisar; o scheduler só coloca o Pod em um nó que tenha esse tanto livre. `limits` é o teto. `500m` significa 500 milicores, meia CPU. Se o processo tentar usar mais CPU que o limite, ele é desacelerado (*throttling*); se passar do limite de memória, o contêiner é encerrado com `OOMKilled`. Neste experimento, cada Pod tem exatamente meia CPU. É isso que torna a capacidade de cada cópia previsível.

Readiness (linhas 32–36). A cada 5 segundos, o kubelet faz um GET em `/readyz`. Enquanto não

responder 200, o Pod não recebe tráfego do Service.

Liveness (linhas 37–42). A cada 10 segundos, um GET em `/healthz`. Se falhar várias vezes seguidas, o kubelet *reinicia* o contêiner. `initialDelaySeconds` dá tempo para o processo subir.

Segurança (linhas 43–46). Recusa rodar como root, impede que o processo ganhe privilégios e deixa o sistema de arquivos do contêiner somente leitura. A aplicação não grava nada em disco, então funciona assim. Muitos clusters de empresa rejeitam Pods que não declaram restrições desse tipo.

```
kubectl apply -f k8s/deployment.yaml
kubectl rollout status deployment/rotas
kubectl rollout history deployment/rotas
kubectl get replicaset
```

O que faz. Aplica a nova versão e acompanha a troca. O `history` agora mostra duas revisões. O `get replicaset` mostra dois ReplicaSets: o novo com 1 Pod e o antigo com 0, guardado para permitir rollback.

Por que agora. Você mudou o template, então o Deployment precisou substituir o Pod. Ele fez isso de forma gradual: subiu o Pod novo, esperou ficar pronto (agora existe readiness!) e só então removeu o antigo.

O que observar. Rode `kubectl describe pod -l app=rotas` e procure `QoS Class`. O valor é `Burstable`, porque o request de memória é menor que o limite. Se requests e limits fossem iguais para CPU e memória, seria `Guaranteed`, a classe menos sujeita a ser despejada quando o nó fica sem memória.

Na entrevista 6

Pergunta: *Qual a diferença entre requests e limits? O que acontece quando um contêiner passa de cada um?*

Uma boa resposta: Requests servem ao agendamento: o scheduler soma os requests e só coloca o Pod onde cabe. Limits são o teto em tempo de execução. Passar do limite de CPU causa throttling (o processo fica mais lento, não morre). Passar do limite de memória faz o kernel encerrar o processo, e o status aparece como `OOMKilled`. Sem requests, o scheduler não tem informação para distribuir Pods e o HPA baseado em CPU não funciona, porque calcula o uso como porcentagem do request.

Na entrevista 7

Pergunta: *Qual a diferença entre readiness e liveness? Pode dar um exemplo de configuração errada?*

Uma boa resposta: Readiness decide se o Pod recebe tráfego; falhar tira o Pod do Service sem reiniciá-lo. Liveness decide se o contêiner precisa ser reiniciado. Um erro clássico é a liveness verificar o banco de dados: se o banco fica lento, todas as réplicas falham a liveness, são reiniciadas ao mesmo tempo e o serviço inteiro cai, sem que reiniciar resolva nada. A liveness deve checar só se o próprio processo está vivo. Neste projeto, as verificações usam endereços leves e separados do cálculo, então um Pod ocupado calculando continua sendo considerado vivo.

Critério para avançar. `kubectl get pods` mostra 1 Pod `1/1 Running`, o `rollout history` tem duas revisões e o `describe` mostra as probes e os recursos.

GRUPO 4

Service

Um endereço que não muda, na frente de Pods que mudam.

Contexto geral. No passo 3.5, o IP do Pod mudou quando ele foi recriado. Com várias réplicas, os IPs mudam a cada escala, atualização ou falha. Um **Service** resolve isso: ele tem um nome DNS e um IP virtual estáveis e mantém uma lista de destinos com os IPs dos Pods que (1) têm as labels do seu `selector` e (2) estão prontos pela readiness. Essa lista fica em objetos chamados **EndpointSlices**. Em cada nó, o `kube-proxy` programa regras de rede para que conexões ao IP do Service sejam encaminhadas para um dos destinos.

A ordem do grupo: escrever e aplicar o Service, conferir a lista de destinos, testar o nome `rotas` de dentro do cluster (o mesmo caminho que o k6 vai usar) e, por último, abrir um acesso a partir do seu computador para usar a interface.

4.1 Escrever e aplicar o Service

Crie `k8s/service.yaml`:

```yaml
apiVersion: v1
kind: Service
metadata:
  name: rotas
  namespace: ops-lab
spec:
  type: ClusterIP
  selector:
    app: rotas
  ports:
    - port: 8000
      targetPort: 8000
```

`type: ClusterIP` cria um endereço acessível só dentro do cluster. O `selector` usa a mesma label do template do Deployment; é ela que liga as duas peças. `port` é a porta do Service; `targetPort` é a porta do contêiner para onde a conexão vai. Aqui são iguais, mas não precisam ser.

```
kubectl apply -f k8s/service.yaml
kubectl get service rotas
```

```
NAME    TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)    AGE
rotas   ClusterIP   10.105.38.211   <none>        8000/TCP   5s
```

4.2 Conferir para onde o Service encaminha

```
kubectl get endpointslices -l kubernetes.io/service-name=rotas
kubectl get pods -o wide
```

O que faz. O primeiro mostra a lista de destinos do Service. O segundo mostra o IP do Pod para você comparar.

Por que agora. Um Service sempre é criado, mesmo com o selector errado. Ele só não terá destinos. Conferir os endpoints é a forma direta de saber se o Service "enxerga" os Pods.

```
NAME          ADDRESSTYPE   PORTS   ENDPOINTS    AGE
rotas-7xq2m   IPv4          8000    10.244.0.7   20s
```

Na entrevista 8

Pergunta: *O Service existe, mas as requisições falham ou dão timeout. Como você investiga?*

Uma boa resposta: Primeiro, `kubectl get endpointslices -l kubernetes.io/service-name=<nome>`. Se não há endpoints, há duas causas comuns: o `selector` do Service não bate com as labels dos Pods (comparo com `kubectl get pods -show-labels`) ou os Pods não estão prontos (coluna READY `0/1`; vejo a readiness no `describe`). Se há endpoints, confiro se `targetPort` é a porta em que o processo realmente escuta, testo de dentro do cluster com um Pod temporário e, se existirem NetworkPolicies, verifico se alguma bloqueia o tráfego.

4.3 Testar o nome rotas de dentro do cluster

```
kubectl run teste --rm -it --restart=Never --image=busybox:1.36 -- wget -qO- http://rotas
    ↪ :8000/healthz
```

O que faz. Cria um Pod temporário com a imagem `busybox` (um Linux mínimo com ferramentas de rede), executa `wget` contra `http://rotas:8000/healthz`, mostra a resposta e apaga o Pod. `-rm` apaga ao terminar, `-it` liga o terminal ao Pod, `-restart=Never` cria um Pod simples em vez de um controlador, e tudo depois de `--` é o comando que roda dentro dele.

Por que agora. É exatamente o caminho que o k6 vai usar: DNS do cluster resolve `rotas`, o Service encaminha, o Pod responde. Se isso não funcionar, não adianta configurar o teste de carga.

O que observar. A resposta é um JSON com `"status": "ok"` e o nome do Pod. O nome curto `rotas` funciona porque o teste está no mesmo namespace. De outro namespace, seria `rotas.ops-lab`; o nome completo é `rotas.ops-lab.svc.cluster.local`. Se aparecer `pod "teste" already exists`, um teste anterior ficou para trás: apague com `kubectl delete pod teste`.

4.4 Acessar a interface pelo seu computador

Abra um **segundo terminal** na mesma pasta e deixe este comando rodando:

```
kubectl port-forward service/rotas 8080:8000
```

O que faz. Cria um túnel da porta 8080 do seu computador até a porta 8000 de *um* Pod escolhido pelo Service no momento em que o túnel abre. O terminal fica ocupado enquanto o túnel existir.

Por que agora. Um Service `ClusterIP` não é acessível de fora do cluster. O `port-forward` é a forma padrão de um profissional acessar algo interno para inspeção, sem expor nada publicamente.

Abra http://localhost:8080, escolha 9 entregas e calcule. A tela mostra a distância, o tempo de cálculo e o Pod que atendeu. Compare o tempo com o `calculo_ms` do Grupo 2: agora o processo tem no máximo meia CPU. Quando terminar, volte ao segundo terminal e encerre com `Ctrl+C`. Isso fecha o túnel, não a aplicação.

Na entrevista 9

Pergunta: *Quais são os tipos de Service e quando usar cada um? Onde entra o Ingress?*

Uma boa resposta: `ClusterIP` (padrão): endereço interno, para comunicação entre serviços. `NodePort`: abre a mesma porta em todos os nós, útil em laboratório ou atrás de um balanceador externo próprio. `LoadBalancer`: pede ao provedor de nuvem um balanceador com IP externo. Ingress (ou o Gateway API, mais recente) não é um tipo de Service: é um roteador HTTP que fica na frente de vários Services e decide pelo host e pelo caminho da URL, geralmente com TLS. Na prática, a maioria dos Services é ClusterIP e só a borda é exposta.

Na entrevista 10

Pergunta: *Abri um `port-forward` para o Service e recarreguei a página várias vezes. Sempre o mesmo Pod respondeu. O balanceamento está quebrado?*

Uma boa resposta: Não. O `port-forward` escolhe um Pod quando o túnel abre e fica preso nele; ele não passa pelo balanceamento do kube-proxy. Para ver a distribuição, é preciso gerar conexões de dentro do cluster ou por um Ingress/LoadBalancer. Além disso, clientes que reaproveitam conexões (HTTP keep-alive, gRPC) também tendem a ficar no mesmo Pod, porque o Service balanceia por conexão, não por requisição.

Critério para avançar. o EndpointSlice lista o IP do Pod, o `wget` de dentro do cluster recebeu `"status": "ok"` e a interface calculou uma rota pelo `port-forward`.

GRUPO 5

Uma implantação que dá errado

Treinar a reversão com calma, antes de precisar dela com pressa.

Contexto geral. Implantações com problema acontecem em qualquer empresa: a pipeline publica uma tag que não chegou ao registry, a nova versão não sobe, uma variável de ambiente está errada. O que diferencia uma operação madura é o tamanho do impacto e o tempo de recuperação.

O Deployment ajuda nos dois. Na estratégia padrão (`RollingUpdate`, com `maxSurge` e `maxUnavailable` de 25%), com uma réplica, o Kubernetes pode criar 1 Pod a mais e não pode ficar com nenhum a menos. Ou seja: o Pod antigo só sai depois que o novo estiver pronto. Se o novo nunca ficar pronto, o antigo continua atendendo. Vamos provocar exatamente isso: pedir uma imagem que não existe.

5.1 Publicar uma versão inexistente

```
kubectl set image deployment/rotas rotas=rotas:1.1
kubectl rollout status deployment/rotas --timeout=60s
```

O que faz. `set image` altera, direto na API, a imagem do contêiner `rotas` do Deployment para `rotas:1.1`, que não existe. O `rollout status` espera até 60 segundos e desiste com erro.

Por que agora. Simula a falha mais comum de pipeline: a tag referenciada não está disponível. Usamos `set image` só para provocar a falha rapidamente; o normal seria mudar o arquivo e aplicar.

5.2 Diagnosticar

```
kubectl get pods
```

```
NAME                     READY   STATUS             RESTARTS   AGE
rotas-58f9c7b6d4-q8wzt   1/1     Running            0          12m
rotas-7c6d5f8b9a-m2hlk   0/1     ImagePullBackOff   0          70s
```

Copie o nome do Pod com problema e leia os eventos dele:

```
kubectl describe pod rotas-7c6d5f8b9a-m2hlk
```

O que observar. Nos Events, a sequência é `Pulling image "rotas:1.1"`, depois `Failed to pull image ... pull access denied` ou `not found`, depois `Back-off pulling image`. `ErrImagePull` é a falha da tentativa; `ImagePullBackOff` é o kubelet esperando cada vez mais entre as tentativas.

Agora confirme que o serviço continua no ar:

```
kubectl run teste --rm -it --restart=Never --image=busybox:1.36 -- wget -qO- http://rotas
    ↪ :8000/healthz
```

Responde normalmente. O Pod novo nunca ficou pronto, então nunca entrou no Service, e o antigo nunca foi removido.

5.3 Reverter e conferir

```
kubectl rollout undo deployment/rotas
kubectl rollout status deployment/rotas
kubectl get pods
kubectl rollout history deployment/rotas
```

O que faz. `rollout undo` volta o template do Deployment para a revisão anterior. O ReplicaSet antigo, que estava guardado, volta a ser o atual, e o Pod com problema é removido.

Por que agora. Em incidente, recuperar o atendimento vem antes de entender a causa. A reversão é rápida porque não depende de reconstruir nada: o ReplicaSet da versão boa já existe.

O que observar. No histórico, a revisão que você restaurou ganha um número novo (a 2 vira 4, por exemplo). O arquivo `k8s/deployment.yaml` continua dizendo `rotas:1.0`, então arquivo e cluster voltaram a concordar.

Na entrevista 11

Pergunta: *Uma implantação em produção começou a gerar erros. O que você faz, nesta ordem?*

Uma boa resposta: Primeiro, limitar o impacto: `kubectl rollout undo` (ou reverter o commit no repositório, se a equipe usa GitOps, para que a ferramenta não reaplique a versão ruim). Segundo, confirmar a recuperação pelos indicadores do serviço, não só pelo status dos Pods. Terceiro, investigar com calma: eventos, logs da versão nova, diferenças de configuração. Para reduzir o impacto da próxima vez: readiness que reflita prontidão de verdade, `maxUnavailable: 0`, publicação canário e anotação `kubernetes.io/change-cause` para o histórico dizer o que mudou em cada revisão. Vale lembrar que o rollback do Deployment não desfaz migrações de banco de dados.

Critério para avançar. `kubectl get pods` mostra só um Pod, `1/1 Running`, com a imagem `rotas:1.0` (confira no `describe`).

GRUPO 6

Teste de carga

Um gerador automático, rodando dentro do cluster, com a mesma demanda em todas as execuções.

Contexto geral. Clicar no navegador não mede capacidade. Vamos usar o **k6**, ferramenta de teste de carga, rodando como um **Job** dentro do próprio cluster. Três motivos para ele rodar lá dentro: (1) ele usa o caminho real, pelo Service, e por isso a carga é distribuída entre os Pods; (2) o túnel do `port-forward` seria um gargalo e prenderia tudo em um Pod; (3) é assim que se testa em empresa, com o gerador perto do serviço para que a rede do seu computador não entre na medição.

Um **Job** é o controlador para tarefas que terminam. O Deployment existe para manter algo rodando para sempre e reinicia o que para; o Job executa até concluir e para. O script do k6 entra no Pod por um **ConfigMap**, objeto que guarda configuração (arquivos ou pares chave–valor) separada da imagem. Assim não precisamos construir uma imagem do k6 só para incluir um arquivo.

A ordem do grupo: entender os parâmetros do script, criar o ConfigMap, escrever o Job, rodar um piloto e calibrar a taxa.

6.1 Entender os parâmetros do teste

Abra `carga.js`. Não é preciso entender o JavaScript; importam as quatro variáveis no topo e o bloco `options`.

Parâmetro	Significado
`TAXA` (padrão 6)	Pedidos *iniciados* por segundo. O executor `constant-arrival-rate` mantém essa taxa mesmo que as respostas demorem, como clientes reais que não esperam os outros terminarem.
`PARADAS` (padrão 9)	Tamanho de cada rota. Fixo em 9 na atividade.
`DURACAO` (padrão 60)	Segundos medidos.
`URL`	Endereço do serviço. O padrão, `http://rotas:8000`, é o nome do Service.
Cenário `aquecimento`	10 s de carga que não entram nas métricas, para estabilizar.
Cenário `medicao`	Começa aos 20 s (10 s de aquecimento e 10 s de intervalo) e dura `DURACAO`.
`noConnectionReuse`	Cada pedido abre conexão nova, então o Service escolhe o Pod a cada pedido.
`thresholds`	Metas: `falhas` abaixo de 1%, p95 de `latencia_ok` abaixo de 800 ms e nenhuma iteração descartada na medição. Se alguma meta falhar, o k6 termina com código de saída 99.

Métricas próprias do script: `rotas_ok` conta rotas válidas na medição; `falhas` é a fração de tentativas que não devolveram rota (inclui os 503); `latencia_ok` é o tempo HTTP só das respostas com sucesso, para que respostas 503 instantâneas não deixem o p95 artificialmente bom.

6.2 Criar o ConfigMap com o script

```
kubectl create configmap carga-script --from-file=carga.js
kubectl describe configmap carga-script
```

O que faz. Cria um ConfigMap chamado `carga-script` com uma chave `carga.js` cujo valor é o conteúdo do arquivo. O `describe` mostra o conteúdo guardado.

Por que agora. O Job vai montar este ConfigMap como um arquivo. Ele precisa existir antes do Job; caso contrário, o Pod do k6 fica preso em `ContainerCreating` com um evento de volume não encontrado.

Gerar o YAML em vez de criar direto. Em empresa, o ConfigMap também iria para o Git. O próprio `kubectl` gera o manifesto sem criar nada:

```
kubectl create configmap carga-script --from-file=carga.js --dry-run=client -o yaml
```

`-dry-run=client` monta o objeto só no seu computador e `-o yaml` imprime. É um atalho comum para começar um manifesto sem decorar todos os campos. O mesmo vale para outros tipos, como `kubectl create deployment ... -dry-run=client -o yaml`.

Na entrevista 12

Pergunta: *Qual a diferença entre ConfigMap e Secret? Secret é criptografado?*

Uma boa resposta: Os dois guardam configuração fora da imagem e podem virar variáveis de ambiente ou arquivos no Pod. Secret é para dados sensíveis (senhas, tokens, certificados), mas por padrão os valores ficam só em base64, que é codificação e não criptografia. A proteção vem de RBAC restringindo quem lê Secrets, da criptografia do etcd em repouso habilitada no cluster e, muitas vezes, de um cofre externo (Vault, gerenciadores de segredos da nuvem) integrado ao cluster. Secret nunca deve ser versionado em texto puro no Git.

6.3 Escrever o Job

Crie `k8s/job-carga.yaml`:

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: carga
  namespace: ops-lab
spec:
  backoffLimit: 0
  activeDeadlineSeconds: 240
  template:
    metadata:
      labels:
        app: gerador-carga
    spec:
      restartPolicy: Never
      containers:
        - name: k6
          image: grafana/k6:1.0.0
          command: ["k6", "run", "/scripts/carga.js"]
          env:
            - name: TAXA
              value: "6"
            - name: PARADAS
              value: "9"
            - name: DURACAO
              value: "60"
          resources:
            requests:
              cpu: 250m
              memory: 128Mi
            limits:
              cpu: 500m
              memory: 256Mi
          volumeMounts:
            - name: script
              mountPath: /scripts
              readOnly: true
      volumes:
        - name: script
          configMap:
            name: carga-script
```

Lendo o arquivo por blocos.

- Linha 7: `backoffLimit: 0` impede que o Job tente de novo se o Pod falhar. Num teste de carga, repetir automaticamente dobraria a carga e misturaria resultados.
- Linha 8: `activeDeadlineSeconds` encerra o Job se ele passar de 4 minutos. É uma proteção contra teste preso consumindo recursos.
- Linha 11–12: label própria, diferente de `app: rotas`. Se o gerador tivesse a label da aplicação, o Service mandaria tráfego para ele.
- Linha 14: `restartPolicy: Never`. Em Jobs, só são aceitos `Never` ou `OnFailure`; o padrão dos Pods, `Always`, não faz sentido para algo que deve terminar.
- Linhas 16–18: imagem oficial do k6, com versão fixa, e o comando que roda o script.
- Linhas 19–25: parâmetros do teste como variáveis de ambiente. Os valores são texto, por isso as aspas.
- Linhas 26–32: o gerador também tem recursos definidos. Ele disputa CPU com a aplicação no mesmo nó; sem limite, poderia distorcer a medição.
- Linhas 33–40: o ConfigMap vira um volume, e o volume é montado em `/scripts`. O arquivo aparece como `/scripts/carga.js`, que é o caminho usado no `command`.

6.4 O ciclo de uma medição

Toda execução do teste segue os mesmos cinco comandos, nesta ordem. Antes do piloto, crie a pasta para os registros:

```
mkdir resultados
```

Ciclo de medição (repita em todo teste)

```
kubectl apply -f k8s/job-carga.yaml
kubectl wait --for=condition=Ready pod -l job-name=carga --timeout=180s
kubectl logs -f job/carga
kubectl logs job/carga > resultados/NOME.txt
kubectl delete job carga
```

Troque `NOME` pelo rótulo do teste: `piloto`, `1pod-a`, `3pods-b` etc.

1. **`apply`** cria o Job, que cria um Pod com o k6.
2. **`wait`** espera o Pod do k6 ficar pronto. O Job adiciona automaticamente a label `job-name=carga` ao Pod, e é por ela que selecionamos. Na primeira vez, a imagem do k6 é baixada; por isso o timeout generoso. Se aparecer `no matching resources found`, o Pod ainda não tinha sido criado: repita o comando.
3. **`logs -f`** acompanha a saída do k6 até o fim (cerca de 90 segundos). O terminal volta sozinho quando o teste termina.
4. **`logs >`** salva a saída completa em arquivo. Os logs pertencem ao Pod; quando o Job for apagado, eles somem. Guardar a evidência antes de apagar é obrigatório.
5. **`delete`** remove o Job e seu Pod. Um Job que terminou não roda de novo, e o template de um Job não pode ser alterado depois de criado. Para a próxima execução, é preciso um Job novo com o mesmo nome; por isso o anterior sai.

Por que esta ordem. Criar, esperar ficar pronto, acompanhar, guardar, limpar. Pular o `wait` pode fazer o `logs -f` desistir enquanto a imagem ainda baixa. Inverter os dois últimos perde os dados.

6.5 Rodar o piloto

Confirme que há uma réplica e rode o ciclo com o nome `piloto`:

```
kubectl get deployment rotas
```

No final do log, o k6 imprime um resumo. As linhas que importam:

```
  ■ THRESHOLDS
    dropped_iterations{scenario:medicao}
    ✓ 'count==0' count=0
    falhas
    × 'rate<0.01' rate=62.41%
    latencia_ok
    ✓ 'p(95)<800' p(95)=512.80ms

  ■ TOTAL RESULTS
    CUSTOM
    falhas.....................: 62.41% 226 out of 362
    latencia_ok................: avg=470.12ms med=468.9ms p(95)=512.80ms max=601.3ms
    rotas_ok...................: 136    1.6/s
```

O que observar. Os valores acima são ilustrativos. O que cada linha significa:

- `rotas_ok`: rotas válidas na janela medida. Para obter **sucessos por segundo**, divida por 60. Não use o `/s` que o k6 mostra ao lado: ele divide pelo tempo total do teste, incluindo aquecimento e intervalo.
- `falhas`: porcentagem e contagem (`226 out of 362` significa 226 falhas em 362 tentativas).
- `latencia_ok p(95)`: 95% das respostas com sucesso levaram até esse tempo.
- `dropped_iterations{scenario:medicao}`: pedidos que o k6 não conseguiu nem iniciar. Precisa ser 0 para a medição representar a taxa pedida.

Depois do `logs -f`, antes de apagar o Job, veja o status:

```
kubectl get job carga
```

Se alguma meta falhou, o status é `Failed`. Isso é esperado com uma réplica e **não** significa que o teste deu errado: o k6 terminou com código 99 para sinalizar meta não atingida, e o Job interpreta qualquer código diferente de zero como falha. O resumo está nos logs normalmente.

Na entrevista 13

Pergunta: *Um Job aparece como `Failed`. Como você sabe se o problema é da aplicação, do teste ou da infraestrutura?*

Uma boa resposta: Olho o código de saída e os logs. `kubectl describe pod -l job-name=<job>` mostra `Last State: Terminated` com `Exit Code`. Para o k6, código 99 significa thresholds não atingidos: o teste rodou e a aplicação não cumpriu as metas, o que é um resultado válido. Se o Pod nem iniciou (`ImagePullBackOff`, volume não encontrado, `Pending`), é infraestrutura. Se aparece `DeadlineExceeded`, o Job passou do `activeDeadlineSeconds`. Se o log não tem resumo, a medição não existe e não deve virar zero na tabela.

6.6 Calibrar e fixar a taxa

Não existe uma taxa certa para todo computador. O objetivo é uma demanda que revele diferença entre as configurações:

- Com 1 Pod, quase tudo teve sucesso? A taxa está baixa: aumente `TAXA` no `job-carga.yaml` (por exemplo, 8) e rode outro piloto (`piloto2`).
- Quase nada teve sucesso e `dropped_iterations` passou de zero? Está alta demais para o seu computador: reduza (por exemplo, 4).
- Houve falhas, mas também uma quantidade razoável de sucessos? Bom ponto de partida.

Uma estimativa ajuda a escolher: se um cálculo leva cerca de 0,45 s com meia CPU, um Pod atende no máximo uns 2 pedidos por segundo. Três Pods, uns 6. Uma taxa entre esses valores costuma ser informativa.

Critério para avançar. um piloto completo foi salvo em `resultados/`, você escolheu a taxa da série e anotou o valor. A partir daqui, `TAXA`, `PARADAS` e `DURACAO` não mudam mais.

GRUPO 7
Escala e medição

Mudar só a quantidade de Pods e repetir o mesmo teste.

Contexto geral. Escalar horizontalmente é mudar `spec.replicas`. O Deployment cria ou remove Pods, o Service passa a incluir ou excluir os novos endereços assim que ficam prontos. O nosso trabalho é garantir que cada medição comece com o cluster estável na quantidade pedida, executar o mesmo ciclo do Grupo 6 e registrar.

Neste grupo usaremos o `kubectl scale`, que é imperativo. É uma escolha consciente para o experimento: ele muda só o número de réplicas, rapidamente e sem tocar no resto do manifesto. O quadro de entrevista do passo 7.1 explica o efeito colateral disso e como as empresas lidam com ele.

7.1 Escalar e esperar a convergência

```
kubectl scale deployment/rotas --replicas=3
kubectl rollout status deployment/rotas
kubectl get pods -o wide
kubectl get endpointslices -l kubernetes.io/service-name=rotas
```

O que faz. `scale` altera o número desejado de réplicas. `rollout status` espera até as três estarem prontas. Os dois últimos confirmam que existem três Pods e que os três estão na lista do Service.

Por que agora. Medir antes da convergência mistura configurações: o teste começaria com uma réplica pronta e duas subindo. Conferir os endpoints garante que o Service já encaminha para todas.

Na entrevista 14

Pergunta: *Você escalou com `kubectl scale -replicas=3`. Depois alguém roda `kubectl apply -f deployment.yaml`, que diz `replicas: 1`. O que acontece? Como evitar?*

Uma boa resposta: O apply leva o Deployment de volta a 1 réplica, porque o arquivo é a fonte da verdade declarada e o `scale` foi uma mudança feita fora dele. Isso é o chamado *drift*. Evita-se mudando o arquivo e aplicando (de preferência por pipeline ou GitOps, com Argo CD ou Flux, que inclusive detectam e corrigem drift). Quando um HPA controla as réplicas, a prática é remover o campo `replicas` do manifesto, para que o apply não brigue com o autoscaler.

7.2 Ler os contadores antes do teste

```
kubectl logs -l app=rotas --prefix --tail=1
```

O que faz. Mostra a última linha de log de cada Pod com a label `app=rotas`. `-prefix` coloca o nome do Pod no começo de cada linha. Como a aplicação escreve os totais acumulados em toda linha, a última linha de cada Pod diz quanto ele já atendeu e recusou desde que iniciou.

Por que agora. Comparando esses totais antes e depois do teste, você descobre como a carga se distribuiu entre os Pods. Pods recém-criados mostram só a linha de início.

7.3 Executar o ciclo e observar os recursos

Rode o ciclo de medição (passo 6.4) com o nome `3pods-a`. Enquanto o `logs -f` roda, use o segundo terminal:

Windows (PowerShell)
Repita a cada 15 s, umas quatro vezes:

```
kubectl top pods
```

Ubuntu (terminal)
Atualização automática a cada 5 s (`Ctrl+C` sai):

```
watch -n 5 kubectl top pods
```

O que faz. Mostra o uso atual de CPU (em milicores) e memória de cada Pod, incluindo o do k6.

O que observar. Pods da aplicação perto de `500m` estão no limite de CPU. Isso é saturação: o sinal de que aquela réplica não consegue fazer mais. Se a medição falhar com `metrics not available yet`, o metrics-server ainda está coletando; espere um pouco. As leituras são amostras de alguns segundos atrás; servem como indício, não como prova.

7.4 Ler a distribuição depois do teste

```
kubectl logs -l app=rotas --prefix --tail=1
```

```
[pod/rotas-58f9c7b6d4-7kd2x/rotas] aceito calculo_ms=441.2 total_aceitos=118 total_recusados=41
[pod/rotas-58f9c7b6d4-q8wzt/rotas] aceito calculo_ms=452.7 total_aceitos=124 total_recusados=37
[pod/rotas-58f9c7b6d4-tz6pn/rotas] aceito calculo_ms=438.9 total_aceitos=115 total_recusados=45
```

Subtraia os valores lidos no passo 7.2. Os números incluem o aquecimento, por isso não batem exatamente com o `rotas_ok` do k6; servem para ver se os três Pods receberam carga parecida. Não precisam ser iguais: o Service escolhe o destino de cada conexão sem saber se o Pod está ocupado.

7.5 Completar a série

Repita cada configuração três vezes, alternando a ordem para que nenhuma seja sempre medida com o computador mais quente ou mais ocupado.

Rodada	Ordem	Réplicas	Nome do teste
A	1	3	`3pods-a` (já feito)
A	2	1	`1pod-a`
A	3	2	`2pods-a`
B	1	2	`2pods-b`
B	2	3	`3pods-b`
B	3	1	`1pod-b`
C	1	1	`1pod-c`
C	2	2	`2pods-c`
C	3	3	`3pods-c`

Para cada linha: `kubectl scale`, `kubectl rollout status`, contadores antes, ciclo de medição, contadores depois. Registre em `resultados_modelo.csv` (abre no Excel, LibreOffice ou Google Planilhas) as colunas pedidas; sucessos por segundo é `rotas_ok` / 60.

Durante a série, não reconstrua a imagem, não mude a taxa e não rode `kubectl apply -f k8s/deployment.yaml` (voltaria a 1 réplica, passo 7.1). Um teste por vez. Teste interrompido não vira zero na tabela: anote o motivo e repita com outro nome.

Critério para avançar. nove medições salvas, todas com a mesma `TAXA`.

GRUPO 8

Diagnóstico, decisão e encerramento

Saber onde procurar quando algo não sai como o esperado, e tirar conclusões que os dados sustentam.

Contexto geral. O último grupo junta três coisas que um profissional faz depois de qualquer mudança: entender os limites do ambiente, interpretar os números e deixar tudo limpo.

8.1 Encontrar o limite do nó

Peça mais réplicas do que cabem:

```
kubectl scale deployment/rotas --replicas=6
kubectl get pods
```

O que faz. Pede 6 Pods de meia CPU cada (3 CPUs só para a aplicação, mais os componentes do cluster). O nó tem 3 CPUs, então parte deles fica `Pending`.

Por que agora. Mostra, com seus próprios olhos, que réplica não é capacidade física. O scheduler soma os requests; se não cabe, não agenda.

Leia os eventos de um Pod `Pending` e a ocupação do nó:

```
kubectl describe pod NOME-DE-UM-POD-PENDING
kubectl describe node ops-lab
```

O que observar. No Pod, um evento `FailedScheduling` com `0/1 nodes are available: 1 Insufficient cpu`. No nó, a seção `Allocated resources` mostra a soma dos requests de CPU perto de 100%. Repare que é a soma dos *requests*, não o uso real: um Pod pode estar pendente mesmo com a CPU ociosa.

Volte ao estado normal:

```
kubectl scale deployment/rotas --replicas=1
```

Na entrevista 15

Pergunta: *Um Pod está em `Pending` há vários minutos. Qual é o seu roteiro de investigação?*

Uma boa resposta: `kubectl describe pod` e leitura dos eventos, que quase sempre dizem o motivo. As causas mais comuns: falta de CPU ou memória nos nós considerando os requests (`Insufficient cpu`), `nodeSelector`, afinidade ou *taints* que nenhum nó satisfaz, PersistentVolumeClaim que não consegue ser vinculado, ou ResourceQuota do namespace estourada (nesse caso o Pod nem é criado e o erro aparece no ReplicaSet). A solução depende do motivo: ajustar requests, liberar ou adicionar nós (ou deixar o Cluster Autoscaler fazer isso), corrigir regras de agendamento.

8.2 Tabela de diagnóstico

Sinal	Onde olhar e o que costuma ser
`Pending`	`describe pod`, Events. Falta de recursos ou regra de agendamento.
`ContainerCreating` demorado	`describe pod`. Imagem baixando ou volume/ConfigMap inexistente.
`ErrImagePull`, `ImagePullBackOff`	`describe pod`. Tag errada, imagem não carregada no nó (`minikube image load`) ou registry inacessível.
`CrashLoopBackOff`	`kubectl logs POD -previous` mostra a saída da execução que caiu. Erro na aplicação, variável faltando, porta já em uso.
`OOMKilled`	`describe pod`, `Last State`. Passou do limite de memória.
`Running` mas `0/1`	Readiness falhando. `describe pod` mostra `Readiness probe failed`.
Service sem resposta	`get endpointslices`. Selector errado ou nenhum Pod pronto.
Job `Failed` com resumo	Metas do k6 não atingidas (saída 99). É resultado, não defeito.
Job sem resumo	Falha de execução. Não é medição.
`metrics not available`	metrics-server ainda iniciando. Aguarde.

Um comando que resume o que aconteceu recentemente no namespace inteiro:

```
kubectl get events --sort-by=.lastTimestamp
```

Na entrevista 16

Pergunta: *Um Pod está em `CrashLoopBackOff`. O que significa e como você investiga?*

Uma boa resposta: O contêiner inicia, termina com erro e o kubelet tenta de novo com espera crescente entre as tentativas. O `kubectl logs` comum pode mostrar a execução atual, que ainda nem escreveu nada; `kubectl logs <pod> -previous` mostra a saída da execução que caiu, onde geralmente está o erro. O `describe` mostra o `Exit Code` e o `Reason` (por exemplo, `OOMKilled` ou `Error`). Causas comuns: configuração ou variável de ambiente faltando, dependência indisponível na inicialização, comando errado na imagem, liveness agressiva demais matando o processo antes de ele terminar de subir.

8.3 Interpretar os resultados

Calcule a mediana de cada configuração (o valor do meio das três repetições ordenadas). A mediana reduz o peso de uma execução muito diferente das outras.

Resultado observado	O que investigar
Mais sucessos e menos falhas com mais Pods	Havia trabalho que podia ser feito em paralelo. Qual é a menor configuração que atende às metas? Quanto custa a mais?
p95 baixo, mas muitas falhas	As recusas foram rápidas. O p95 dos sucessos não representa quem ficou sem atendimento.
Pouca diferença entre 1 e 3 Pods	A taxa pode estar baixa demais, ou o nó (ou o próprio k6) pode estar limitado. Veja o `kubectl top`.
Resultado pior com 3 Pods	Disputa por CPU no nó, carga do gerador, outras tarefas no computador.
Um Pod recebe quase tudo	Readiness de algum Pod falhando ou conexões sendo reaproveitadas. Confira endpoints e contadores.
`dropped_iterations` maior que zero	O k6 não gerou toda a demanda. A medição não representa a taxa pedida.

Por que três Pods não acabam com as falhas? O Service escolhe o destino de cada conexão sem saber se o Pod está ocupado. Com três Pods e uma taxa próxima da capacidade total, um pedido ainda pode cair em um Pod que está calculando e receber 503, mesmo havendo outro livre. Um balanceador que

conhece a ocupação, uma fila ou um algoritmo mais eficiente mudariam esse comportamento. Isso não é uma limitação do Kubernetes; é uma consequência de como a aplicação foi desenhada (um cálculo por Pod, sem fila).

Na entrevista 17

Pergunta: *Como você configuraria o Kubernetes para escalar esta aplicação automaticamente?*

Uma boa resposta: Com um HorizontalPodAutoscaler. Ele precisa do metrics-server (ou outra fonte de métricas) e de requests de CPU definidos, porque calcula a utilização como porcentagem do request. Exemplo: `kubectl autoscale deployment rotas -cpu-percent=70 -min=1 -max=3`. Neste laboratório faríamos isso só depois de medir: o experimento manual é o que diz qual é a capacidade de uma réplica e qual é um máximo razoável. Sem esse conhecimento, o autoscaler só automatiza um chute. Também lembraria que o HPA leva algum tempo para reagir, então picos muito rápidos ainda precisam de folga ou de fila.

8.4 Escrever a análise

Entregue, junto com o CSV preenchido e a pasta `resultados/`, uma análise curta com quatro partes:

1. **Cenário.** Computador usado, recursos do cluster, taxa escolhida e por quê, duração, recursos por Pod.
2. **Evidências.** Tabela com as medianas de 1, 2 e 3 Pods: sucessos por segundo, falhas, p95, iterações descartadas. Distribuição entre Pods e uso de CPU observado.
3. **Explicação.** Por que mais réplicas ajudaram, ajudaram pouco ou pioraram. Separe o que foi observado do que é hipótese.
4. **Decisão.** Qual configuração você adotaria para essa demanda e quanto ela custa em CPU reservada. Se nenhuma atingiu as metas, qual seria a próxima investigação.

Limite da conclusão

Três Pods em um único nó não protegem contra a queda desse nó. Este laboratório mede atendimento sob carga; não testa recuperação de desastre. Em produção, as réplicas seriam distribuídas entre nós e zonas.

8.5 Encerrar

```
kubectl delete namespace ops-lab
```

O que faz. Apaga o namespace e, em cascata, tudo o que está dentro: Deployment, ReplicaSets, Pods, Service, ConfigMap, Jobs.

Por que agora. Mostra por que agrupar os objetos da atividade em um namespace facilita a limpeza. É também o comando mais perigoso deste roteiro: em um cluster compartilhado, conferir o contexto antes dele não é opcional. Os manifestos continuam na pasta `k8s/`; para recriar tudo, basta `kubectl apply -f k8s/namespace.yaml` seguido de `kubectl apply -f k8s/deployment.yaml -f k8s/service.yaml` e o ConfigMap.

Para liberar os recursos do computador:

```
minikube stop
```

`stop` desliga o cluster e preserva seu estado; `minikube start -p ops-lab` liga de novo. Para apagar o cluster de vez, `minikube delete -p ops-lab`.

APÊNDICE A

Referência rápida, na ordem do roteiro

Estrutura de um comando kubectl. `kubectl VERBO TIPO/NOME [-n NAMESPACE] [opções]`. Verbos de leitura: `get`, `describe`, `logs`, `top`. Verbos que alteram: `apply`, `create`, `delete`, `scale`, `set`, `rollout`. Seletores: `-l chave=valor`. Formatos de saída: `-o wide`, `-o yaml`, `-o json`. Para consultar os campos de qualquer objeto sem sair do terminal:

`kubectl explain deployment.spec.template.spec.containers`

```
# Grupo 1: cluster e contexto
minikube start -p ops-lab --driver=docker --cpus=3 --memory=3072
minikube profile ops-lab
kubectl config current-context
kubectl get nodes -o wide
kubectl get pods -A
kubectl apply -f k8s/namespace.yaml
kubectl config set-context --current --namespace=ops-lab
minikube addons enable metrics-server

# Grupo 2: imagem
docker build -t rotas:1.0 .
docker run -d --rm -p 8000:8000 --name rotas-teste rotas:1.0
docker logs rotas-teste
docker stop rotas-teste
minikube image load rotas:1.0

# Grupo 3: deployment
kubectl apply -f k8s/deployment.yaml
kubectl rollout status deployment/rotas
kubectl get deployment,replicaset,pods
kubectl describe pod NOME
kubectl logs deployment/rotas
kubectl delete pod -l app=rotas
kubectl rollout history deployment/rotas

# Grupo 4: service
kubectl apply -f k8s/service.yaml
kubectl get endpointslices -l kubernetes.io/service-name=rotas
kubectl run teste --rm -it --restart=Never --image=busybox:1.36 -- wget -qO- http://rotas:8000/healthz
kubectl port-forward service/rotas 8080:8000

# Grupo 5: falha e rollback
kubectl set image deployment/rotas rotas=rotas:1.1
kubectl rollout status deployment/rotas --timeout=60s
kubectl rollout undo deployment/rotas

# Grupo 6: teste de carga
kubectl create configmap carga-script --from-file=carga.js
kubectl apply -f k8s/job-carga.yaml
kubectl wait --for=condition=Ready pod -l job-name=carga --timeout=180s
kubectl logs -f job/carga
kubectl logs job/carga > resultados/NOME.txt
kubectl delete job carga

# Grupo 7: escala e medição
kubectl scale deployment/rotas --replicas=3
kubectl logs -l app=rotas --prefix --tail=1
kubectl top pods

# Grupo 8: diagnóstico e encerramento
kubectl get events --sort-by=.lastTimestamp
kubectl describe node ops-lab
kubectl delete namespace ops-lab
minikube stop
```

APÊNDICE B

Deployment completo (segunda versão)

Use para conferir o seu arquivo depois do passo 3.6.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: rotas
  namespace: ops-lab
  labels:
    app: rotas
spec:
  replicas: 1
  selector:
    matchLabels:
      app: rotas
  template:
    metadata:
      labels:
        app: rotas
    spec:
      containers:
        - name: rotas
          image: rotas:1.0
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8000
          resources:
            requests:
              cpu: 500m
              memory: 64Mi
            limits:
              cpu: 500m
              memory: 128Mi
          readinessProbe:
            httpGet:
              path: /readyz
              port: 8000
            periodSeconds: 5
          livenessProbe:
            httpGet:
              path: /healthz
              port: 8000
            initialDelaySeconds: 10
            periodSeconds: 10
          securityContext:
            runAsNonRoot: true
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
```

O que seria diferente em produção. A imagem viria de um registry, com tag ligada ao commit. Os manifestos estariam no Git e seriam aplicados por pipeline ou GitOps, provavelmente organizados com Kustomize ou Helm para variar valores por ambiente. Haveria pelo menos duas réplicas distribuídas entre nós, um PodDisruptionBudget, exposição por Ingress ou Gateway com TLS, métricas e logs coletados continuamente e alertas baseados nos objetivos de serviço. A aplicação em si usaria um servidor HTTP adequado e um algoritmo de roteirização que não fosse força bruta.

Documentação oficial (consultada em 15 out. 2026)

- kubectl Quick Reference e kubeconfig e contextos.
- Kubernetes: Deployment, Service, Job, ConfigMap, recursos e probes.
- Kubernetes: depuração de Pods.
- Minikube: instalação e como levar imagens ao cluster.
- k6: taxa de chegada constante, thresholds e iterações descartadas.